

Demerits of MongoDB

1. Being a NoSQL database, it does not follow the principles of ACID (atomicity, consistency, isolation and durability). Therefore, it cannot be used in applications that need transactions at the database level or where ACID compliance is necessary.
2. It does not allow the use of stored procedures.

Trying out MongoDB

We will not be discussing how to install MongoDB in this article. So, let us start with creating a database and tables, and look at a few common use cases.

Type the following command to start MongoDB:

```
net start MongoDB
```

To run the MongoDB shell, type:

```
mongo
```

Type the command below to create a database:

```
use database_name
```

...where *database_name* represents the name of your intended database; e.g.:

```
use firstdatabase
```

Creating a collection

There are two methods of creating a collection.

1. *Creating a collection when creating a document*
You can insert a document in a non-existent collection, and MongoDB will automatically create the collection.
The syntax is as follows:

```
db.collection_name.insert({key:value,  
key:value...})
```

Example:

```
> use firstdatabasedb  
switched to db firstdatabasedb  
  
db.firstdatabase.insert({  
  name: "Charles",  
  age: 27,  
  website: "firstdatabase.com"  
})
```

The above will create a collection name, *firstdatabase*, and then insert the specified values. To check if the document was successfully inserted, run the following command:

```
db.collection_name.find()
```

To check for collection, run the following command:

```
show collections
```

2. *Creating a collection before the documents*

Here, you create the collection first and then you can insert documents into it. The syntax for this is shown below:

```
db.createCollection(name, options)
```

name represents the collection, whereas *options* is used to specify parameters.

Example:

```
> db.createCollection("employees")  
{ "ok" : 1 }
```

Basic data manipulation methods are:

- Insert document
- Use of *insert()*

The syntax for this is:

```
db.collection_name.insert()
```

As an example, we insert a document in a collection named *first collection*:

```
db.firstcollection.insert(  
  {  
    name: "Charles",  
    age: 27,  
    email: "admin@firstcollection.com",  
    course: [ { name:  
      "Databases", duration: 7 }, { name:  
      "Datastructures", duration: 30 } ]  
  }  
)
```

The *insert()* method is multifunctional as it can create a collection even if it does not exist. The course parameter above has several values inserted.

Inserting several documents in a collection

To do this, we first define documents and then use the *insert()* method, as shown below. To insert two documents in a collection named *employees*, type:

```
var beginners =  
[  
  {  
    "EmployeeId" : 001,  
    "EmployeeName" : "Ludacris",  
    "age": 29  
  },  
  {  
    "EmployeeId" : 002,  
    "EmployeeName" : "Nelly",  
    "age": 36  
  },  
];
```

```
db.students.insert(beginners);
```

Query document

We will use *find()* to query documents in a collection.

Querying documents in the JSON format: If we have a collection called *employees* in a database *employeesdb*, to get documents from it, use the command below:

```
db.employees.find()
```